

BI Entscheidungsagent Online Retail II

*Interaktives Dashboard mit regelbasiertem KI-Entscheidungsagenten
für einen UK-basierten Online-Händler*

Student	Sheyla Sampietro
Kurs	Business Intelligence
Dozent	Manuel Renold
Abgabedatum	April 2026
GitHub	github.com/SheylaSam/entscheidungsagent-bi
Dataset	Online Retail II (UCI ML Repository)

1. Inhaltsverzeichnis

- 1 Projektübersicht und Zielsetzung
- 2 Dataset — Online Retail II
- 3 Technologie-Entscheide
- 4 Systemarchitektur
- 5 Datenverarbeitung und Bereinigung
- 6 Analyse-Module
- 7 KI-Entscheidungsagent
- 8 Streamlit Dashboard
- 9 Testing und Qualitätssicherung
- 10 Fazit und Ausblick

2. Projektübersicht und Zielsetzung

Dieses Projekt implementiert ein vollständiges Business-Intelligence-System für einen UK-basierten Online-Händler (Geschenkartikel). Das System besteht aus drei Kernkomponenten: einer Datenpipeline, die Rohdaten aus Excel in eine relationale Datenbank überführt; einer Analyse-Schicht, die Umsatzprognosen, Kundensegmentierung und Produktperformance berechnet; sowie einem regelbasierten KI-Entscheidungsagenten, der aus diesen Analysen konkrete Managemententscheide ableitet.

Das Projektziel lässt sich in einem Satz zusammenfassen: *Aus 1,07 Millionen Transaktionen automatisch priorisierte Handlungsempfehlungen für das Management generieren.*

Anforderungen

Anforderung	Umsetzung
Datenbankanbindung	SQLite-Datenbank, automatisch aus Excel befüllt beim ersten Start
Interaktives Dashboard	Streamlit-App mit 5 Tabs, Plotly-Diagramme
KI-Entscheidungsagent	Regelbasierter Agent kombiniert Forecast + RFM + Produktdaten
Dokumentation	Dieser Bericht + README mit Tech-Entscheiden + GitHub Repo
Reproduzierbarkeit	<pre>pip install -r requirements.txt && streamlit run app.py</pre> genügt

3. Dataset — Online Retail II

Das verwendete Dataset stammt vom UCI Machine Learning Repository und beschreibt alle Transaktionen eines UK-basierten Online-Händlers für Geschenkartikel im Zeitraum Dezember 2009 bis Dezember 2011.

Eigenschaft	Wert
Quelle	UCI ML Repository — Online Retail II
Zeitraum	01.12.2009 – 09.12.2011
Gesamtzeilen (roh)	~1'067'000 Transaktionszeilen
Sheets	Year 2009–2010 (~525k), Year 2010–2011 (~542k)
Spalten	8 (Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer ID, Country)
Primäres Segment	B2B-Grosshändler und -Händler (UK)

Datenqualität und Bereinigung

Das Rohdataset enthält verschiedene Anomalien, die vor der Analyse entfernt werden müssen:

- **Stornierungen:** Invoice-Nummern beginnend mit «C» (negative Quantities) — werden vollständig entfernt
- **Fehlende Customer IDs:** ~25% der Zeilen enthalten keine Kundennummer — für RFM nicht verwendbar, werden gefiltert
- **Negative/Null-Preise:** Testbestellungen und interne Buchungen mit $\text{Price} \leq 0$ — werden entfernt
- **Testcodes:** StockCodes wie «POSTAGE», «BANK CHARGES», «M» — nicht-produktbezogene Einträge

Nach der Bereinigung bleiben ca. 750'000–800'000 verwertbare Transaktionszeilen für die Analyse.

FEATURE ENGINEERING

Nach der Bereinigung wird eine `revenue`-Spalte berechnet: `revenue = quantity × price`. Diese Spalte ist die Grundlage für alle monetären Analysen (KPIs, RFM Monetary, Produktumsatz, Forecast).

4. Technologie-Entscheide

Für dieses Projekt wurden zwei bewusste Abweichungen vom Kurs-Standard vorgenommen, die hier dokumentiert und begründet werden.

Python + Streamlit statt R + Shiny

ENTSCHEID

Der Kurs hat primär R und Shiny für interaktive Dashboards eingeführt. Für dieses Projekt wurde bewusst Python + Streamlit gewählt.

Kriterium	Python + Streamlit	R + Shiny
KI-Integration	✅ Prophet, scikit-learn, pandas sind Python-nativ	⚠ Nur via <code>reticulate</code> — fehleranfällig
Pipeline-Konsistenz	✅ Eine Sprache für alles (ETL, ML, Dashboard)	❌ Hybrid nötig: R für UI, Python für ML
Kurskonsistenz	✅ Week 08: Python (<code>SimpleAutoencoder.py</code> , Prophet-Guide)	✅ Primärsprache des Kurses
Lernkurve	✅ Geringerer Boilerplate, intuitive Reaktivität	⚠ Reactive-Programmiermodell komplex
Portfolio-Relevanz	✅ Industrie-Standard für Data Science Dashboards	⚠ Hauptsächlich akademisch/statistisch

R + Shiny wäre die richtige Wahl gewesen, wenn die KI-Komponente entfallen würde und das Projekt rein deskriptiv-statistischer Natur wäre. Da die Prophet-Zeitreihe und die RFM-Berechnung den Kern des Projekts bilden, ist Python die einheitlichere Wahl.

SQLite statt MySQL

ENTSCHEID

Der Kurs verwendete MySQL (inkl. `MysqlConnector.py`). Für dieses Projekt wurde SQLite gewählt.

Kriterium	SQLite	MySQL
Portabilität	✅ Eine einzige <code>.db</code> -Datei, kein Server nötig	❌ Server-Installation + Konfiguration
Reproduzierbarkeit	✅ Automatisch beim ersten Start generiert	❌ Manueller Import, Credentials nötig
Scope-Angemessenheit	✅ Read-only BI-Dashboard, kein Concurrent Write	⚠️ Mehrwert ohne Concurrent Access nicht vorhanden
SQL-Kompatibilität	✅ Standard SQL, identisches Abfragemodell	✅ Vollständiges RDBMS

Das Prinzip (SQL-Queries, relationales Modell, Datenbankabstraktionsschicht) ist in beiden Systemen identisch — nur die Engine ist bei SQLite leichtgewichtiger. Für ein lokales BI-Dashboard ohne gleichzeitige Schreibzugriffe mehrerer Nutzer ist SQLite die angemessene Wahl.

Verwendeter Tech Stack

Schicht	Technologie	Zweck
Rohdaten	Excel (online_retail_II.xlsx)	Quelldaten (UCI ML Repository)
Datenbank	SQLite (retail.db)	Persistenz, SQL-Abfragen
Datenverarbeitung	Python + Pandas	ETL, Bereinigung, Feature Engineering
Visualisierung	Plotly	Interaktive Charts
Dashboard	Streamlit	Web-UI, 5-Tab-Struktur
Forecasting	Prophet (Meta)	Bayesianische Zeitreihenprognose
Testing	pytest	Automatisierte Unit-Tests
Versionierung	GitHub	Source Control, Portfolio

5. Systemarchitektur

Excel (online_retail_II.xlsx) | | einmaliger Import beim ersten Start ▼
SQLite Datenbank (retail.db) | Tabelle: transactions | Spalten: invoice,
stock_code, description, | quantity, invoice_date, price, | customer_id,
country, revenue | | SQL-
Queries | ▼ ▼ ▼ Prophet RFM Produkt- Forecasting Segmentierung Analyse
(forecasting.py) (rfm_analysis.py) (product_analysis.py) | | |
| | | | ▼ KI-Entscheidungsagent
(decision_agent.py) | ▼ Streamlit Dashboard (app.py)
| | Tab 1: Übersicht (KPIs) | | Tab 2: Forecast |
| Tab 3: Kunden RFM | | Tab 4: Produkte | | Tab 5: KI-Entscheid |

Abbildung 1: Systemarchitektur des BI Entscheidungsagenten

Modulstruktur

Datei	Verantwortlichkeit
src/data_processing.py	Excel-Import, Bereinigung, SQLite-Befüllung. Wird einmalig ausgeführt, wenn retail.db noch nicht existiert.
src/rfm_analysis.py	RFM-Berechnung pro Kunde (Recency / Frequency / Monetary), Quintil-Scoring, Segment-Zuweisung.
src/forecasting.py	Monatsaggregation, Prophet-Fit, 3-Monats-Prognose mit Konfidenzintervall.
src/product_analysis.py	Top-10-Produkte nach Umsatz, Erkennung von Produkten mit ≥3 aufeinanderfolgenden Monatsrückgängen.
src/decision_agent.py	Regelbasierter Agent: kombiniert alle drei Analysen und erzeugt priorisierte Empfehlungen.
app.py	Streamlit-Hauptapplikation mit 5 Tabs und @st.cache_data -Caching aller Datenladeoperationen.

Datenbankschema

Die Datenbank enthält eine einzige Tabelle `transactions`, die alle bereinigten Transaktionen speichert:

```
CREATE TABLE transactions (  
    invoice      TEXT,  
    stock_code   TEXT,  
    description  TEXT,  
    quantity     REAL,  
    invoice_date TEXT,  -- ISO-Format: YYYY-MM-DD HH:MM:SS  
    price        REAL,  
    customer_id  TEXT,  
    country      TEXT,  
    revenue      REAL  -- Computed: quantity × price  
);
```

Das Single-Table-Design ist für dieses Projekt bewusst gewählt: Die Quelldaten sind denormalisiert (jede Zeile enthält alle Informationen), und für ein Read-only BI-Dashboard sind keine Schreiboperationen oder normalisierten Relationen nötig. Sämtliche Aggregationen werden durch SQL-Queries und Pandas direkt auf dieser Tabelle durchgeführt.

6. Datenverarbeitung und Bereinigung

Das Modul `src/data_processing.py` implementiert die vollständige ETL-Pipeline (Extract, Transform, Load) vom Excel-Rohdatensatz zur SQLite-Datenbank.

ETL-Pipeline

Extract

Beide Excel-Sheets («Year 2009-2010» und «Year 2010-2011») werden mittels `pandas.read_excel()` mit dem Parameter `sheet_name=None` eingelesen und zu einem DataFrame konkateniert. Da der Excel-Import bei 1 Million Zeilen mehrere Minuten dauern kann, wird die Datenbank nur beim ersten Start generiert — bei allen folgenden Starts wird die vorhandene `retail.db` direkt verwendet.

Transform — Bereinigungsschritte

```
def clean_dataframe(df):
    df = df.rename(columns=COLUMN_MAP)           # Spaltennamen normalisieren
    df = df[df['quantity'] > 0]                  # Stornierungen entfernen
    df = df[df['price'] > 0]                     # Null/Negativpreise entfernen
    df = df.dropna(subset=['customer_id'])       # Zeilen ohne Kundennummer
    df['revenue'] = df['quantity'] * df['price'] # Revenue berechnen
    return df
```

Load

Der bereinigte DataFrame wird über `pandas.DataFrame.to_sql()` in die SQLite-Datenbank geschrieben. Bei einem Re-Run wird die bestehende Tabelle ersetzt (`if_exists='replace'`).

Einstiegspunkt

Die Funktion `build_database()` wird beim Start der Streamlit-App automatisch aufgerufen:

```
def build_database(excel_path=EXCEL_PATH, db_path=DB_PATH):  
    if db_path.exists():  
        return # DB bereits vorhanden – kein Re-Import  
    df = pd.concat([pd.read_excel(excel_path, sheet_name=s)  
                    for s in ['Year 2009–2010', 'Year 2010–2011']])  
    clean_df = clean_dataframe(df)  
    load_to_sqlite(clean_df, db_path)
```

PERFORMANCE-HINWEIS

Der erste Start dauert 2–5 Minuten (abhängig von Hardware), da ~1 Million Excel-Zeilen verarbeitet werden. Streamlit zeigt während des Imports einen Spinner. Alle nachfolgenden Starts laden direkt aus der SQLite-Datenbank und sind in Sekunden bereit.

7. Analyse-Module

6.1 Prophet-Zeitreihenprognose

Das Modul `src/forecasting.py` erstellt eine 3-Monats-Umsatzprognose mithilfe des Prophet-Bibliothek von Meta (ehemals Facebook). Prophet ist ein additives Zeitreihenmodell, das automatisch Trend, wöchentliche und jährliche Saisonalität erkennt und bei fehlenden Werten robust ist.

Datenvorbereitung

Die Transaktionsdaten werden auf Monatsebene aggregiert. Prophet erwartet zwei Spalten: `ds` (Datum) und `y` (Wert):

```
SELECT DATE(invoice_date, 'start of month') as month,
       SUM(revenue) as revenue
FROM transactions
GROUP BY month
ORDER BY month
```

Modellierung und Prognose

```
from prophet import Prophet

model = Prophet(yearly_seasonality=True, weekly_seasonality=False)
model.fit(series)  # series: DataFrame mit ds/y-Spalten
future = model.make_future_dataframe(periods=3, freq='MS')
forecast = model.predict(future)
```

Das Modell gibt für jeden Monat drei Werte zurück: `yhat` (Punktprognose), `yhat_lower` und `yhat_upper` (80%-Konfidenzintervall). Im Dashboard werden historische Werte als Linie und die Prognosewerte als schraffiertes Band dargestellt.

6.2 RFM-Kundensegmentierung

Das Modul `src/rfm_analysis.py` segmentiert alle Kunden nach dem RFM-Modell (Recency, Frequency, Monetary). RFM ist ein etabliertes Verfahren im CRM und BI, um Kunden anhand ihres Kaufverhaltens zu klassifizieren.

RFM-Metriken

Metrik	Definition	Bedeutung
Recency (R)	Tage seit dem letzten Kauf	Niedriger Wert = aktiver Kunde
Frequency (F)	Anzahl Bestellungen	Hoher Wert = treuer Kunde
Monetary (M)	Gesamtumsatz pro Kunde	Hoher Wert = wertvoller Kunde

Quintil-Scoring

Jede Metrik wird in 5 Quintile eingeteilt (Score 1–5). Für Recency gilt: Score 5 = kürzlicher Kauf (niedrige Tage), Score 1 = lange nicht gekauft. Für Frequency und Monetary: Score 5 = höchster Wert.

Segmente

Segment	RFM-Regel	Bedeutung
Champions	R=5, F≥4	Beste Kunden: kürzlich, häufig, hoher Umsatz
Loyal	F≥3, R≥3	Regelmässige Käufer mit guter Recency
At Risk	R≤2, F≥3	Früher aktiv, jetzt abgewandert — Reaktivierung nötig
Lost	R=1, F≤2	Inaktiv, kaum Kaufhistorie — schwer zurückzugewinnen
New	R≥4, F=1	Neue Kunden — Bindung aufbauen
Others	Kein Match	Mittelsegment ohne klares Profil

6.3 Produkt-Performance

Das Modul `src/product_analysis.py` liefert zwei Analysen:

Top-10-Produkte

Aggregation des Gesamtumsatzes je Produkt (StockCode + Description), Auswahl der 10 grössten Positionen. Dargestellt als horizontales Balkendiagramm im Dashboard.

Rückläufige Produkte

Für jeden Monat wird der Monatsumsatz je Produkt berechnet. Anschliessend wird geprüft, ob der Umsatz über ≥ 3 aufeinanderfolgende Monate konsistent gesunken ist. Diese Produkte gelten als «rückläufig» und werden dem Entscheidungsagenten übergeben.

```
def get_declining_products(monthly_df, months=3):
    # Für jedes Produkt: prüfe ob die letzten `months` Monate
    # jeweils niedriger als der Vormonat waren
    declining = []
    for product, grp in monthly_df.groupby('description'):
        last_n = grp.sort_values('month').tail(months + 1)['revenue'].values
        if all(last_n[i] > last_n[i+1] for i in range(len(last_n)-1)):
            declining.append(product)
    return declining
```

8. KI-Entscheidungsagent

Das Herzstück des Projekts ist der regelbasierte Entscheidungsagent in `src/decision_agent.py`. Er kombiniert die Ergebnisse aller drei Analyse-Module (Forecast, RFM, Produktanalyse) und leitet daraus priorisierte Managemententscheide ab.

Design-Entscheid: Regelbasiert statt LLM

Für BI-Dashboards in der Praxis ist ein regelbasierter Agent einer LLM-Integration in mehreren Punkten überlegen:

- **Transparenz:** Jede Empfehlung ist direkt auf konkrete Metriken und Schwellwerte zurückführbar — kein «Black-Box»-Verhalten

- **Determinismus:** Gleiche Datenlage ergibt immer die gleiche Empfehlung — reproduzierbar und auditierbar
- **Keine Latenz/Kosten:** Kein API-Call nötig, Empfehlungen werden in Millisekunden generiert
- **Kurskonformität:** Der Fokus liegt auf der BI-Methodik, nicht auf dem Prompt-Engineering

Entscheidungsregeln

REGEL 1 — REAKTIVIERUNGSKAMPAGNE **HOCH**

Bedingung: Forecast nächster Monat < letzter Istwert um >5% **UND** At-Risk-Kunden-Anteil >20%

Entscheid: Sofortige Reaktivierungskampagne für At-Risk-Segment lancieren

Logik: Rückläufiger Umsatz kombiniert mit einer grossen Gruppe abgewandelter Kunden signalisiert ein akutes Churn-Problem, das durch gezielte Retention-Massnahmen (z.B. Rabattcodes, persönliche E-Mails) adressiert werden kann.

REGEL 2 — SORTIMENTSBEREINIGUNG **MITTEL**

Bedingung: Mindestens 1 Produkt mit ≥ 3 aufeinanderfolgenden Monatsrückgängen

Entscheid: Rückläufige Produkte aus dem Sortiment nehmen oder repositionieren

Logik: Ein anhaltender Umsatzrückgang über 3+ Monate ist kein saisonales Muster, sondern ein strukturelles Problem (sinkende Nachfrage, Qualitätsprobleme, Wettbewerb).

REGEL 3 — KEIN HANDLUNGSBEDARF **TIEF**

Bedingung: Keine der obigen Regeln greift

Entscheid: Aktuelle Strategie beibehalten

Logik: Positive oder stabile Forecasts ohne Kundenabwanderung und ohne rückläufige Produkte indizieren, dass kein kurzfristiger Handlungsbedarf besteht.

Empfehlungsformat

Jede Empfehlung wird als strukturiertes Dictionary zurückgegeben:

```
{  
  "priority":  "HOCH",          # HOCH / MITTEL / TIEF  
  "finding":   "Forecast -8% ...", # Was die Daten zeigen  
  "decision":  "Reaktivierungskampagne ...", # Konkrete Massnahme  
  "reasoning": "Forecast: -8%, At-Risk: 23% ..." # Welche Metriken  
}
```

Mehrere Regeln können gleichzeitig zutreffen. In diesem Fall werden alle aktiven Empfehlungen zurückgegeben, nach Priorität absteigend sortiert.

9. Streamlit Dashboard

Das Frontend ist als Streamlit-Applikation mit 5 Tabs implementiert. Alle Dateladeoperationen sind mit `@st.cache_data` dekoriert, sodass die SQLite-Queries und Prophet-Berechnungen nur einmal pro Session ausgeführt werden.

Tab-Struktur

Tab	Inhalt	Zielgruppe
1 — Übersicht	4 KPI-Kacheln (Umsatz, Kunden, Bestellungen, Forecast-Delta), Mini-Trendchart, Kundensegment-Balken, kompakte KI-Alert-Box	Management, Executive View
2 — Forecast	Historischer Monatsumsatz als Linie, Prophet-Prognose mit Konfidenzband, 3 Forecast-Metriken als Kacheln	Vertrieb, Controlling
3 — Kunden RFM	Scatter Plot (Recency vs. Frequency, Grösse = Monetary), Segmentzusammenfassung als Tabelle, At-Risk-Kundenliste	Marketing, CRM
4 — Produkte	Top-10 horizontales Balkendiagramm, Tabelle rückläufiger Produkte	Category Management, Einkauf
5 — KI- Entscheid	Farbkodierte Empfehlungskarten (Rot=HOCH, Gelb=MITTEL, Grün=TIEF), vollständige Begründung, Entscheidungslogik-Tabelle	Management, Entscheidungsträger

Technische Implementierung

Caching

```
@st.cache_data
def load_data():
    conn = get_connection()
    rfm      = load_rfm(conn)
    actuals, forecast = load_forecast(conn)
    top_prod, declining = load_product_analysis(conn)
    recs      = generate_recommendations(forecast, rfm, declining)
    return rfm, actuals, forecast, top_prod, declining, recs
```

Prioritäts-Farbkodierung (Tab 5)

```
PRIORITY_COLORS = {
    'HOCH':  ('#fee2e2', '#991b1b'),  # Rot
    'MITTEL': ('#fef9c3', '#854d0e'),  # Gelb
    'TIEF':  ('#dcfce7', '#166534'),  # Grün
}

for rec in recommendations:
    bg, fg = PRIORITY_COLORS[rec['priority']]
    st.markdown(f"""
    <div style="background:{bg}; border-left: 4px solid {fg}; padding:16px;">
        <strong style="color:{fg}">{rec['priority']}: {rec['decision']}</strong>
        <p>{rec['reasoning']}</p>
    </div>""", unsafe_allow_html=True)
```

10. Testing und Qualitätssicherung

Das Projekt verwendet Test-Driven Development (TDD) mit pytest. Alle fünf Module haben dedizierte Testdateien. Insgesamt sind 22 Tests implementiert, die vollständig auf synthetischen In-Memory-Daten laufen — keine echte Datenbank oder Excel-Datei nötig.

Testübersicht

Testdatei	Anzahl Tests	Abgedeckte Fälle
tests/test_data_processing.py	6	Bereinigung, Stornierungen, fehlende IDs, Revenue-Berechnung, DB-Schreiben/Lesen
tests/test_rfm_analysis.py	5	RFM-Berechnung, Segmentzuweisung (Champions, At Risk, Lost, New), Scoring
tests/test_forecasting.py	3	Monatsserie-Aggregation, Prophet-Forecast Ausgabestruktur, Prognosehorizont
tests/test_product_analysis.py	4	Top-N-Selektion, rückläufige Produkte (3-Monats-Regel), Grenzfälle
tests/test_decision_agent.py	4	Regel 1 (HOCH), Regel 2 (MITTEL), Regel 3 (TIEF), Mehrfachregeln
Total	22	

Testausführung

```
$ pytest tests/ -v
===== test session starts =====
tests/test_data_processing.py ..... [ 6/22]
tests/test_rfm_analysis.py ..... [11/22]
tests/test_forecasting.py ... [14/22]
tests/test_product_analysis.py .... [18/22]
tests/test_decision_agent.py .... [22/22]
===== 22 passed in 4.32s =====
```

DESIGN-ENTSCHEID: IN-MEMORY TESTS

Alle Tests verwenden synthetische DataFrames und SQLite-In-Memory-Datenbanken (`sqlite3.connect(':memory:')`). Dies macht die Tests unabhängig vom echten Dataset (das aus Grösse-Gründen nicht ins Repo eingchecked wird) und ermöglicht schnelle, isolierte Ausführung.

11. Fazit und Ausblick

Ergebnis

Das Projekt hat alle Anforderungen vollständig umgesetzt: Die Datenpipeline überführt 1,07 Millionen Rohtransaktionen automatisch in eine SQLite-Datenbank; drei Analyse-Module (Prophet-Forecast, RFM-Segmentierung, Produktanalyse) liefern betriebswirtschaftlich relevante Erkenntnisse; der regelbasierte Entscheidungsagent fasst diese zu priorisierten Managemententscheiden zusammen; das 5-Tab-Streamlit-Dashboard visualisiert alle Resultate interaktiv.

Das Projekt ist vollständig reproduzierbar: Nach dem Klonen des Repositorys und dem Download des Datasets genügt ein einziger Befehl (`streamlit run app.py`), um das Dashboard zu starten — ohne Server, ohne manuelle Datenbankkonfiguration, ohne weitere Setup-Schritte.

Erkenntnisse

- **Datenqualität ist entscheidend:** ~25–30% der Rohtransaktionen werden in der Bereinigung entfernt (Stornierungen, fehlende Kundennummern, Testeinträge). Ohne diese Bereinigung würden alle Analysen verzerrte Resultate liefern.
- **Prophet ist saisonal robust:** Das Modell erkennt die Weihnachts-Saisonalität des Geschenkartikel-Shops automatisch und berücksichtigt sie in der Prognose.
- **RFM ist simpel aber wirksam:** Mit nur drei Metriken und einem Quintil-Scoring lassen sich Kunden sinnvoll für gezielte Massnahmen segmentieren.
- **Regelbasierter Agent genügt für BI:** Für strukturierte Management-Entscheide auf der Grundlage klarer Metriken ist ein LLM nicht nötig — Transparenz und Determinismus sind wichtiger als Flexibilität.

Mögliche Erweiterungen

- **Länder-Filter:** Dashboard-Filter nach Country (UK, Germany, France etc.) für regionalen Vergleich
- **Zeitraum-Slider:** Interaktiver Datumsfilter für historische Vergleiche
- **E-Mail-Alert:** Automatischer Versand der KI-Empfehlung bei Überschreiten der Schwellwerte

- **Erweiterter Agent:** Zusätzliche Regeln (z.B. Länder-basierte Empfehlungen, saisonale Produktvorschläge)
- **Cloud-Deployment:** Deployment auf Streamlit Community Cloud für öffentlichen Zugang

REPOSITORY

Der vollständige Quellcode, die Testdateien, dieses Dokument und eine detaillierte Setup-Anleitung sind auf GitHub verfügbar:

<https://github.com/SheylaSam/entscheidungsagent-bi>